

Week 1 Application Exercises

Excel and SQLite Projects · version 2026-09-02

You will work through six exercises. Three of them are spreadsheet exercises done in Excel. Three of them are database exercises done in SQLite, a database engine that runs from a single file — and in this edition, that file lives inside a web page in your browser. Where data has to move from a text file into a database, a short Python script does the importing, and it runs in the browser too.

Everything you need is in this folder. Each chapter has its own page: open it in your browser and the instructions, the SQL boxes, the Terminal, and your database are all there. You do not need the textbook to complete these exercises, you do not need Microsoft Access, and you do not need to install anything.

The six exercises

Chapter	Spreadsheet exercise (Excel)	Database exercise (on the page)
1	1-34 Ticket Sales at Campus Travel — <code>TicketSales.csv</code>	1-35 Tracking Frequent-Flier Miles at Campus Travel — <code>FrequentFliers.txt</code>
2	2-36 Valuing Information Systems — <code>TC0.csv</code>	2-37 Designing the Frequent-Flier Database — no data file; you create it
3	3-40 Tracking Frequent-Flier Mileage — <code>frequentflier2.xlsx</code>	3-41 Building a Knowledge Database — <code>employees.db</code>

Each exercise is worth 100 points and all six are graded the same way; the table is at the end of this handout.

Before you start: setting up

You need two things.

- **A laptop or Chromebook with a current Chrome or Edge.** Firefox and Safari should also work; phones and iPads are not supported for the database exercises. You need internet access the first time each page is opened, so that the page can download its database engine — and, the first time you use it, Python.
- **Excel** for the spreadsheet exercises. Your student Microsoft 365 account includes Excel. LibreOffice Calc or Google Sheets will also work for every spreadsheet exercise here.

Python and DB Browser for SQLite are now optional fallbacks: nothing here asks you to install them. If you would rather work outside the browser, or a page cannot download its engine where you are, DB Browser for SQLite (sqlitebrowser.org) opens the same `.db` files the pages make, and Python 3 (python.org/downloads) runs `load_data.py` from a terminal window exactly as the page does. Both are free and run on Windows, macOS, and Linux.

Folder layout

Unzip the starter files into a folder called `Week 1`. You should have:

- Week 1/
 - `week-1-chapter-1.html` — the Chapter 1 page (Exercises 1-34 and 1-35)
 - `week-1-chapter-2.html` — the Chapter 2 page (Exercises 2-36 and 2-37)
 - `week-1-chapter-3.html` — the Chapter 3 page (Exercises 3-40 and 3-41)
 - `load_data.py`
 - `chapter-01/`
 - `data/TicketSales.csv`
 - `data/FrequentFliers.txt`
 - `chapter-02/`
 - `data/TC0.csv`
 - `chapter-03/`
 - `data/frequentflyer2.xlsx`
 - `data/employees.csv`
 - `employees.db`

The three `.html` files are the homework pages, one per chapter; this handout is in the folder as a PDF as well. The data files are also built into each page — the Files list in the Database panel shows the same tree, with a Download button on every file — so a page keeps working if it is moved on its own. The copies in the folder are the ones you open in Excel.

How the pages work

Open a chapter's page in your browser. At the top is a box for your name and a row of buttons. Below that comes the spreadsheet exercise, then the Database panel, then the database exercise, with the grading table and a Getting unstuck list at the end. Nothing is uploaded: the page downloads a small database engine the first time it opens and after that does all of its work on your computer.

Every SQL step has its own box with a Run SQL button. Each time you press Run, the page opens your database file, runs your SQL from top to bottom, and saves the file again. The results appear under the box and stay there, so what you see is what Export my work will write out. The Database panel lists every table with its row count and updates after every run; Browse shows the first rows of a table and Structure shows how it was created — the same "did the import work" check you would do in any database tool.

The Terminal in the Database panel is where `load_data.py` runs, with the same command you would type in a terminal window on your own computer. A step that needs it shows the command next to a button that puts it in the Terminal for you; you press Run and read what it prints.

Your work and your database are kept inside this browser on this computer. They are not kept if you use a different browser or computer, and any other local web page opened in this browser can see them, so on a shared computer: Export, Download, then Clear my work before you leave.

Three buttons in the toolbar matter for handing in. Download gives you your `.db` file. Export my work writes the document you hand in — your name, every box with its result, the Terminal transcript, and a summary of your database — so press Run on every box before you export. Print / Save PDF makes a paper copy for your

own notes. Reset database and Clear my work are at the bottom of the Database panel, and both ask before they do anything.

Where Python comes in

Python appears in two places on the pages, and only the first is required.

load_data.py

This one script replaces the Access import wizard. You give it three things — a data file, a name for the table, and the database file to put it in — and it creates the table and loads every row:

```
python load_data.py <data file> <table name> <database file>
```

It reads the header row, converts each heading into a safe column name (Meal Category becomes meal_category), inspects the values to decide whether each column should be INTEGER, REAL, or TEXT, creates the table, and inserts the rows. Then it prints the schema it built and the row count so you can confirm the import before you write a query. You run it in the page's Terminal; the first time, the page downloads Python (about 12 MB, once) and shows its progress — later runs are quick. Press View next to load_data.py in the Files list and read the comments: you are expected to understand roughly what it is doing, not to have written it.

The Python cell

Each page also has one Python cell, for exploring on your own. It starts with a few lines that read your database with Python's sqlite3 module and print what they find; nothing graded asks you to write Python. Two things are different from Python on your own computer: input() is not available on the page — put the value directly in your code instead — and there is no pip. Everything the assignment needs (sqlite3, csv) is already there. Importing an .xlsx file needs openpyxl, which means a locally installed Python 3; no exercise requires it.

Chapter 1

Two exercises from Chapter 1. Exercise 1-34 is a spreadsheet exercise done in Excel. Exercise 1-35 is a database exercise done right here on this page: the database engine (SQLite) and the small Python import script both run inside your browser, so there is nothing to install. Your written answers go in the boxes on this page.

Exercise 1-34 • Spreadsheet Application: Ticket Sales at Campus Travel

SCENARIO

You are the office manager at Campus Travel. Your salespeople have logged every ticket sale for the first quarter, and you need a summary of the quarter for the regional manager.

DATA

chapter-01/data/TicketSales.csv — 35 sales, each with a date, a salesperson, a destination, and a number of tickets sold. Four label cells near the bottom of the sheet are waiting for your answers. The file is in your Week 1 folder, and it is also in the Files list on this page if you need another copy.

STEPS

- 1 Open `TicketSales.csv` in Excel and immediately save it as `TicketSales.xlsx`. Working in `.xlsx` means your formulas and formatting survive; a `.csv` file keeps neither.
- 2 Fill in the four summary cells next to the labels already in the sheet, using a formula in each one — not a number you worked out yourself:

The four summary cells and the function each one uses

Label	Function to use
Total Tickets Sold	SUM
Most Tickets Sold	MAX
Least Tickets Sold	MIN
Average Number of Tickets Sold	AVERAGE

- 3 Format the **Date Sold** column as a date and widen every column so no value is cut off or showing as `####`.
- 4 Add a summary table on the same sheet, to the right of the data, showing total tickets sold by each salesperson. Use `SUMIF` (or a PivotTable if you prefer). List the six salespeople and their totals.
- 5 Add a second summary table showing total tickets sold to each destination, again with `SUMIF` or a PivotTable.
- 6 Build a bar chart from your salesperson summary. Give it a title that says what it shows, and label the axes.
- 7 Sort the sales data by **Salesperson** (A to Z), then by **Date Sold** (oldest first). Be careful to sort the data rows only — do not drag the summary label rows into the sort.

SUBMIT

`TicketSales.xlsx`, with formulas intact. Your written answers go in the boxes on this page and are included when you Export my work.

ONE THING TO NOTICE

Your **AVERAGE** will not be a whole number. Think about what a fractional ticket means, and whether “average tickets per sale” is the number the regional manager actually wants, or whether “average tickets per salesperson per month” would tell them more. You will be asked about this kind of distinction all term.

Notice Two or three sentences: which average would the regional manager actually want, and why?

Exercise 1-35 · Database Application: Tracking Frequent-Flier Miles at Campus Travel

SCENARIO

Campus Travel keeps its frequent-flier customer list in a text file that gets passed around by email. Every time someone wants an answer — how many customers request kosher meals, which airline has the most enrolled customers — they scroll through the file by hand. You are going to put it in a database so those questions can be answered with a query instead.

DATA

`chapter-01/data/FrequentFliers.txt` — 43 customers, tab-delimited (each value is separated from the next by a tab character), with name, address, city, state, ZIP, phone, frequent-flier number, airline, meal category, and seating preference.

This page starts with no database. Step 1 creates `campus_travel.db`; from then on the [Database panel](#) shows it, and the Download button in the toolbar saves it to your computer.

STEPS

- 1 In the Terminal, run the import below. Read the schema it prints — it is part of your export. You should see 43 rows loaded into a table called `frequent_fliers`.

The script that does the importing, `load_data.py`, has a View button in the Files list. Read its comments: you are expected to understand roughly what it is doing, not to have written it.

expect: Loaded 43 rows

```
$ python load_data.py chapter-01/data/FrequentFliers.txt frequent_fliers
campus_travel.db
```

- 2 In the [Database panel](#), click **Browse** on `frequent_fliers` and confirm the customers are there. This is the same “did the import work” check you would do in Access.
- 3 Look carefully at the column types the script chose. `zip` came in as `INTEGER`. In one or two sentences, explain why that is a poor choice for ZIP codes in general, even though it happens to cause no damage with this particular data. (Hint: think about a customer in Boston, ZIP 02134.)

- 4 Re-import the file, this time forcing the identifier columns to text. In the Terminal, run the command below, then confirm in the printed schema that both columns are now `TEXT`. Each import rebuilds the table from the text file, so anything you change in the database later (step 6) is undone if you run an import again.

expect: Loaded 43 rows · **expect:** zip TEXT

```
$ python load_data.py chapter-01/data/FrequentFliers.txt frequent_fliers
campus_travel.db --text-columns zip frequent_flier_number
```

- 5** In the SQL boxes below, write and run a query for each of the following. Each box keeps its query and its result, and both go into your export.
- 5a** Every customer who requests a vegan meal, showing first name, last name, and airline, sorted by last name.
- expect:** 13 rows
- SQL box on the web page (step 5a).
- 5b** A count of customers in each meal category, highest count first. Your result should have four rows.
- expect:** 4 rows
- SQL box on the web page (step 5b).
- 5c** A count of customers per airline, highest first.
- expect:** 4 rows
- SQL box on the web page (step 5c).
- 5d** Every customer in the state of Idaho who prefers a window seat, showing name, city, and phone number.
- expect:** 1 row
- SQL box on the web page (step 5d).
- 5e** A count of customers per seating preference.
- expect:** 3 rows
- SQL box on the web page (step 5e).
- 6** Query (e) exposes a data-quality problem: one of the seating values is misspelled in the source file, and it appears in 6 rows. Find it, then fix it in the database with an `UPDATE` statement. Re-run query (e) to confirm the fix. Put both the `UPDATE` statement and the re-run of query (e) in this box, so the corrected result is saved with it.
- expect:** OK · 6 rows changed · **expect:** 3 rows again
- SQL box on the web page (step 6).
- 7** Write one query of your own that answers a question a Campus Travel manager would plausibly ask, using `GROUP BY`. State the question in plain English above the SQL: put your question on the first line as a SQL comment (`-- Which city ...?`) and the query underneath.
- expect:** starts with a `--` comment
- SQL box on the web page (step 7).

Explore Optional — read the same database from Python. Everything on this page is a normal SQLite file, so Python can open it too. This snippet only reads; it does not change your database. Put it in the Python cell and press Run Python, then try changing the column it groups by.

```
import sqlite3
conn = sqlite3.connect('campus_travel.db')
for row in conn.execute('SELECT state, COUNT(*) FROM frequent_fliers GROUP BY
state'):
    print(row)
conn.close()
```

SUBMIT

campus_travel.db (use Download in the toolbar), plus ch1-queries.md, made by Export my work — it contains the printed schema, your answer to step 3, every query with its result, and your step-7 question and query.

Getting unstuck · Chapter 1

The misspelling came back

The re-import in step 4 rebuilds the table from the text file; run your step-6 UPDATE again.

Chapter 2

Two exercises. In 2-36 you build a total-cost-of-ownership model in Excel. In 2-37 you design a three-table frequent-flier database from scratch: the CREATE TABLE statements, the test data, the constraint tests and the queries all run in the SQL boxes on this page, and the database file you build is yours to download.

Exercise 2-36 • Spreadsheet Application: Valuing Information Systems

SCENARIO

Your organization runs a main campus, two branch campuses, and two small stations. You have been handed the annual cost of the network infrastructure broken out by site and by category, and asked to work out the total cost of ownership so leadership can decide where consolidation would save the most money.

DATA

chapter-02/data/TC0.csv — nine cost categories down the side, five sites across the top. Some cells are empty on purpose: not every site has every cost. The file is in your Week 1 folder, and you can also download it from the Files list further down this page.

STEPS

- 1 Open the file in Excel and save it as `TC0.xlsx`.
- 2 Fill in the **TOTAL** column so each cost category shows its total across all five sites.
- 3 Fill in the **TOTAL** row so each site shows its total across all nine categories.
- 4 Put the grand total in the cell where that row and column meet. Then check your work: the sum of the row totals and the sum of the column totals must be the same number. If they are not, you have missed a cell.
- 5 The empty cells matter. `SUM` treats a blank as zero, which is what you want here. In one sentence, say what would go wrong if those blanks held the text `"N/A"` instead.

Step 5 — what would go wrong if the blanks held "N/A"?

- 6 Format every currency cell with a dollar sign, thousands separators, and no decimal places.
- 7 Add a column headed **% of Total** showing what share of the grand total each cost category represents. Use an absolute reference (`$`) for the grand total so the formula fills down correctly, and format the column as a percentage with one decimal place.
- 8 Sort the cost categories from most expensive to least expensive.
- 9 Build a pie chart of cost by category and a column chart of cost by site.

- 10** Write two or three sentences answering: **which single category dominates this budget, and what does that suggest about where consolidation would actually save money?** Support your answer with your own numbers.

Step 10 — which category dominates, and where would consolidation save money?

SUBMIT

TC0.xlsx, with formulas and both charts. Your written answers go in the boxes on this page and are included when you Export my work.

Exercise 2-37 · Database Application: Designing the Frequent-Flier Database

SCENARIO

In Chapter 1 you loaded the frequent-flier list into a single flat table, which is exactly how it arrived. That table repeats the airline name on every row that uses it, and offers no place to record an individual flight. Campus Travel now wants to track actual mileage, so the design has to change. You will build the database from scratch this time, writing the `CREATE TABLE` statements yourself.

DATA

None supplied — you are designing the structure and entering a small amount of test data of your own. This page starts with no database. Its database is `mileage.db`: the first time a `CREATE TABLE` in step 2 succeeds, the page makes the file and the [Database panel](#) starts listing your tables. Until then the panel and the Download button say there is no database yet.

STEPS

- 1 This page's database is `mileage.db`. There is nothing to create by hand: your first successful `CREATE TABLE` statement in step 2 creates it. Open the [Database panel](#) now and notice that it is empty; come back to it after each step to watch the tables and their row counts appear.

2

In the SQL box below, write `CREATE TABLE` statements for three tables:

airlines

Column	Type	Notes
airline_id	INTEGER	primary key
airline_name	TEXT	required, and no two airlines may share a name
alliance	TEXT	e.g. Star Alliance, SkyTeam, Oneworld

customers

Column	Type	Notes
customer_id	INTEGER	primary key
last_name	TEXT	required
first_name	TEXT	required
phone_number	TEXT	
frequent_flier_number	TEXT	no two customers may share one
airline_id	INTEGER	foreign key referencing airlines

flights

Column	Type	Notes
flight_id	INTEGER	primary key
customer_id	INTEGER	foreign key referencing customers
flight_date	TEXT	store as YYYY-MM-DD
origin	TEXT	three-letter airport code
destination	TEXT	three-letter airport code
miles	INTEGER	required, and must be greater than zero

Use `PRIMARY KEY`, `NOT NULL`, `UNIQUE`, `CHECK`, and `FOREIGN KEY` where the notes call for them. Create `airlines` first, then `customers`, then `flights`, because each one refers to the one before it.

Keep the three `DROP TABLE IF EXISTS` lines at the top of the box (`flights`, then `customers`, then `airlines`). Re-running this box rebuilds the three tables from empty, so if you change it later, run step 3 again afterwards. Your finished `CREATE TABLE` statements stay in this box and go into `ch2-database.md` when you Export my work.

expect: 3 tables in the Database panel · **expect:** 0 rows each until step 3

SQL box on the web page (step 2).

- 3 Insert 4 airlines, 6 customers spread across those airlines, and at least 15 flights spread across those customers, using `INSERT INTO`. Make the data plausible — real airline names, real airport codes, dates within the last year. Include one customer who has enrolled but has not flown yet — you will need them in step 5b. That customer can be one of the six, or a seventh.

Insert the airlines first, then the customers, then the flights, in the same order you created the tables. After each Run, check the row counts in the [Database panel](#) and click **Browse** on a table to see the rows.

To run this box again — for example after adding the customer who has not flown — run step 2 first (it empties the three tables), then this box. Run on its own it stops at `UNIQUE constraint failed`, because the rows are already there.

expect: 4 airlines · **expect:** at least 6 customers · **expect:** at least 15 flights · **expect:** after any re-run of step 2, run this box again

SQL box on the web page (step 3).

- 4 Prove your constraints work. Write an `INSERT` that should fail, run it, and record the error message SQLite gives you. Then say in one sentence which constraint stopped it and why that is a good thing. Good candidates: a flight with `miles of -100`, a second customer with a frequent-flier number already in use, or a flight pointing at a `customer_id` that does not exist.

One failing statement per Run — the page stops at the first error. Keep the one you are writing about in this box. The error message is saved with the box and goes into your export; the sentence goes in the written-answer box underneath.

Note: SQLite does not enforce foreign keys unless you turn them on. If your “failing” insert did not fail, it is now in your table. Add `PRAGMA foreign_keys = ON;` above the `INSERT` and delete the stray row (`DELETE FROM flights WHERE customer_id = 999;`) before moving on. Put the `DELETE` on its own Run, or above the `INSERT` — anything below a failing statement never runs. Each Run opens your database fresh, so the `PRAGMA` line must be in the same box as the `INSERT` it protects.

The optional second box is for one more failing `INSERT` if you want to test a second constraint — the same rules apply.

This is the step that separates designs; take it seriously even if it takes a few tries.

expect: 1 INSERT · **expect:** the output is an error message, not a result

SQL box on the web page (step 4).

Step 4 — which constraint stopped it, and why is that a good thing?

Optional second SQL box on the web page (step 4).

- 5 Write and run these queries:

- 5a** Every customer with the name of their airline, using a `JOIN`. Sort by last name.
expect: one row per customer · **expect:** airline name, not airline_id
SQL box on the web page (step 5a).
- 5b** Total miles flown by each customer, most miles first. Use `JOIN`, `GROUP BY`, and `SUM`. Include customers with no flights at all — that will require a `LEFT JOIN`.
expect: one row per customer · **expect:** the customer with no flights shows 0, not missing
SQL box on the web page (step 5b).
- 5c** Total miles flown on each airline.
expect: 4 rows
SQL box on the web page (step 5c).
- 5d** Every flight over 1,000 miles, showing the customer's name, the date, the route, and the mileage.
expect: flights over 1,000 miles only · **expect:** name, date, route, miles
SQL box on the web page (step 5d).
- 5e** The single customer who has flown the most miles.
expect: 1 row
SQL box on the web page (step 5e).
- 6** Compare this design with the flat table from Exercise 1-35. In one short paragraph, name **two specific things this three-table design lets you do that the flat table could not, and one thing that got harder**.

Step 6 — two things this design lets you do, and one thing that got harder

--

Explore Optional. Python can add many rows at once with `executemany()`. This example writes to a separate practice file, `scratch.db`, never to `mileage.db`, so nothing you built for 2–37 changes. Put it in the Python cell and press Run Python. Afterwards the [Database panel](#) lists `scratch.db` as a second file; leave `mileage.db` selected for the SQL boxes.

```
# Optional: Python can add many rows at once with executemany().
# This writes to scratch.db, a separate practice file. It never touches
# mileage.db, so nothing you built for 2–37 changes.
import sqlite3
conn = sqlite3.connect('scratch.db')
conn.execute('DROP TABLE IF EXISTS airports')
conn.execute('CREATE TABLE airports (code TEXT PRIMARY KEY, city TEXT)')
rows = [
    ('PUW', 'Pullman'),
    ('SEA', 'Seattle'),
    ('LHR', 'London Heathrow'),
]
conn.executemany('INSERT INTO airports VALUES (?, ?)', rows)
conn.commit()
for row in conn.execute('SELECT * FROM airports ORDER BY code'):
    print(row)
conn.close()
```

SUBMIT

`mileage.db` (press **Download mileage.db** in the toolbar), plus `ch2-database.md`, made by **Export my work**, which contains your `CREATE TABLE` statements, your failed-insert evidence from step 4, all five queries with results, and your step-6 paragraph. Your written answers go in the boxes on this page and are included when you Export my work.

Getting unstuck · Chapter 2

UNIQUE constraint failed: airlines.airline_id

You re-ran the step-3 `INSERT` box on rows that are already in the tables. Run step 2 first — it rebuilds the three tables from empty — then run step 3 again.

Chapter 3

Two exercises. The first is a spreadsheet lookup done in Excel. The second is a knowledge database done in SQLite, and the database runs right here on this page — there is nothing to install. `employees.db` is already open in the Database panel below; when you are done, download it and export your work.

Exercise 3-40 • Spreadsheet Application: Tracking Frequent-Flier Mileage

SCENARIO

The mileage figures for the quarter have arrived from the airlines, but they came as a separate list keyed by frequent-flier number rather than by name. You need one sheet that shows each customer alongside their miles.

DATA

chapter-03/data/frequentflier2.xlsx — two worksheets. **Customers** has 43 customers with an empty **Miles Flown** column. **Miles Flown** has 43 frequent-flier numbers with their mileage, in a different order. The workbook is in your Week 1 folder; you can also download it from the [Files list](#) further down this page.

STEPS

- 1 Open the workbook. Look at both sheets and identify the column they have in common. That shared column is what makes the lookup possible.
- 2 On the Customers sheet, fill the Miles Flown column using `VLOOKUP` (or `XLOOKUP`) against the Miles Flown sheet. Use an absolute reference for the lookup range so the formula fills down correctly, and ask for an exact match — the last argument must be `FALSE` (or `0`).
- 3 Confirm every row found a match. Wrap your formula in `IFERROR` to display `Not found` instead of `#N/A`, then check that the phrase appears nowhere in the column.
- 4 Below the data, add these summary figures, each with a formula:
 - total miles flown by all customers
 - average miles per customer
 - highest and lowest mileage
 - the count of customers who flew more than 5,000 miles (use `COUNTIF`)
- 5 Add a column headed **Status** that shows `Gold` for 7,500 miles or more, `Silver` for 5,000 to 7,499, and `Standard` below 5,000. Use nested `IF` statements or `IFS`.
- 6 Apply conditional formatting to the Miles Flown column so that Gold-level values stand out.
- 7 Sort the customers by miles flown, highest first, and identify the top five.
- 8 Answer in two or three sentences: Campus Travel wants to offer a bonus to its top tier. Based on your Status column, roughly what share of customers would qualify, and is that the right size for a rewards tier?

SUBMIT

frequentflier2.xlsx with your formulas, the summary figures, the Status column, and the conditional formatting in place. Your written answers go in the boxes on this page and are included when you Export my work.

Exercise 3-41 • Database Application: Building a Knowledge Database

SCENARIO

Campus Travel has offices in four towns. When a customer calls with a detailed question about a particular airline — its baggage rules, its upgrade policy, its alliance partners — whoever picks up the phone needs to find the colleague who actually knows that airline. Right now that means asking around. You are going to turn the staff directory into a searchable knowledge database, so the question “who here knows British Airways?” is a query rather than a group email.

DATA

`chapter-03/employees.db` — a ready-made SQLite database with an `employees` table: 24 staff members, each with a last name, first name, office, area of expertise, and home phone number. This page has already opened it for you — it is the database shown in the [Database panel](#). (`chapter-03/data/employees.csv` holds the same data as text. If you break `employees.db`, **Reset database** restores it; rebuilding with `load_data.py` replaces the table and removes any column you added in step 5.)

STEPS

- 1 In the [Database panel](#), click **Structure** on the `employees` table to see how the table is built, then **Browse** to see the 24 employees. This is the same “is the data really there?” check you would do in any database tool.
- 2 Write and run a query for each of the following, each in its own SQL box. Each box keeps its SQL and its last result, and both go into your export.
- 2a Every employee whose expertise is British Airways, showing first name, last name, office, and phone number, sorted by last name.
expect: 7 rows
SQL box on the web page (step 2a).
- 2b A count of employees per office, largest office first.
expect: 4 rows · **expect:** Pullman 12
SQL box on the web page (step 2b).
- 2c A count of employees per area of expertise.
expect: 4 rows
SQL box on the web page (step 2c).
- 2d Everyone in the Pullman office, sorted by expertise then last name.
expect: 12 rows
SQL box on the web page (step 2d).

- 2e** Every employee whose expertise is Delta or Continental, in a single query. Use `IN` rather than two conditions joined by `OR`.
- expect:** 10 rows
- SQL box on the web page (step 2e).
- 2f** A breakdown of expertise within the Pullman office — which airlines are covered there, and by how many people.
- expect:** 4 rows
- SQL box on the web page (step 2f).
- 3** Now the question the knowledge database exists to answer. A customer calls the Lewiston office with a detailed British Airways question. Write one query that returns the people who could take that call, and write two or three sentences explaining what your result tells you about staffing at that office. The query goes in the SQL box; your sentences go in the answer box under it.
- expect:** one query · **expect:** then 2–3 sentences about staffing at that office
- SQL box on the web page (step 3).
-
- 4** Find the coverage gaps. Write a query that shows which office/expertise combinations do *not* exist — that is, for each office, which of the four airlines nobody there covers. A `CROSS JOIN` of the distinct offices and the distinct expertise values, with a `LEFT JOIN` back to `employees` and a `WHERE ... IS NULL`, will get you there. This is the hardest query in the assignment; attempt it seriously even if you do not finish it.
- expect:** 4 rows
- SQL box on the web page (step 4).
- 5** The table has no primary key, and two employees could share a name. Write an `ALTER TABLE` statement that adds an `employee_id` column, then explain in one or two sentences why a table like this should have had one from the start. After you run it, click **Structure** in the Database panel to confirm the column is there — your downloaded `employees.db` will include it.
- expect:** `employee_id` appears in Structure · **expect:** then 1–2 sentences
- SQL box on the web page (step 5).
-

- 6 Write two or three sentences on the following: this database makes it easy to find who knows what. What does it *not* capture that a real knowledge management system would need? Think about how expertise is recorded here, and who decides what goes in that column.

ExploreOptional — the same kind of question, asked from Python. This code opens `employees.db`, counts the staff for each area of expertise, and prints one line per airline. It only reads the file; it changes nothing. Put it in the Python cell in the Database panel and press **Run Python**. Compare the output with your result from 2c.

```
import sqlite3
conn = sqlite3.connect('chapter-03/employees.db')
for row in conn.execute('SELECT expertise, COUNT(*) FROM employees GROUP BY
expertise ORDER BY 2 DESC, 1'):
    print(row)
conn.close()
```

SUBMIT

`employees.db` with your changes — press **Download employees.db** in the toolbar — plus `ch3-queries.md`, which **Export my work** writes for you, containing every query with its result, your answers to steps 3, 5, and 6, and your best attempt at step 4.

What to hand in

Create one folder per chapter, put the finished files for that chapter in it, and zip each folder on its own:

One zip per chapter

Zip	Contents
Chapter1.zip	TicketSales.xlsx · campus_travel.db · ch1-queries.md (made by Export my work)
Chapter2.zip	TC0.xlsx · mileage.db · ch2-database.md (made by Export my work)
Chapter3.zip	frequentflyer2.xlsx · employees.db · ch3-queries.md (made by Export my work)

Upload each zip to the matching Excel and SQLite Projects item for Week 1.

The `.db` file comes from the Download button in the page's toolbar. The `.md` document comes from Export my work: it holds your name, the schema printed in the Terminal, every query with its result, and your written answers, which go in the boxes on the page — so press Run on every box before you export, and open the file to check it shows your results before you zip it.

Save your spreadsheets as `.xlsx`, not `.csv` — a `.csv` file cannot hold a formula, and formulas are most of what is being graded. Submit the `.db` files themselves, not screenshots of them.

How each exercise is graded

Points per exercise

Criterion	Points
Data imported correctly, row counts confirmed	15
Formulas and queries produce correct results	40
Formulas used where a formula was asked for (not typed-in values)	15
Formatting, sorting, charts, and labeling	10
Written answers show reasoning, not just a restated result	20
Total per exercise	100

Two notes on grading. A spreadsheet where the right number was typed into the cell by hand scores zero on the formula line even though the number is correct — the formula is the skill. And the written answers are worth a fifth of the grade; a one-line answer that restates the number you just calculated will not earn them.

Getting unstuck

The page says SQLite (or Python) could not be downloaded

the page needs an internet connection the first time it opens. Check your connection, then press Retry. If it says the file on the CDN does not match this page's build, tell your professor; until it is fixed, DB Browser for SQLite with the files in the Files list is the offline alternative.

The first Python run is slow

the page is downloading Python (about 12 MB), once. Wait until the Database panel says "Python ready"; later runs take a moment, not a minute. Do not reload the page while it is downloading.

no such table

the import has not been run in this browser yet, you pressed Reset database, or your import wrote to a differently named `.db` file (for example `campustravel.db` when the boxes use `campus_travel.db`). Run the import again with the exact command from the step, or pick the right file in the Database panel.

table already exists

your `CREATE TABLE` has already run once. Put `DROP TABLE IF EXISTS` above it, as Exercise 2-37 step 2 shows, and run the box again.

Every query suddenly returns 0 rows

you re-ran step 2 of Exercise 2-37, which rebuilds the three tables from empty. Run your step 3 `INSERT` s again.

no such column

the importer cleaned the names: `Meal Category` became `meal_category`. Click Structure in the Database panel to see the exact column names.

A query returns nothing

check your text is spelled and capitalised exactly as it is in the data (Browse the table). `WHERE expertise = 'British airways'` matches nothing; `'British Airways'` matches seven people. `WHERE expertise LIKE 'british%'` is a way to sidestep this while you are exploring. If the spelling is right, an empty result is still a result: it means nobody matches, and saying so is a legitimate answer.

My work is gone

your work lives in this browser on this computer. A different browser, a different computer, a private window, or clearing the browser's site data starts you from a blank page. Export my work and Download your `.db` often, and always before you leave a shared computer — then Clear my work.

I opened the .db somewhere else

Download gives you a standard SQLite file: DB Browser for SQLite opens it, and so does Python's `sqlite3`. To bring a file back into the page, use Open a `.db` file... in the toolbar.

input() is not available

the page cannot pause Python to ask you for keyboard input. Put the value directly in your code instead.

My Python INSERT, UPDATE or DELETE did not stick

Python's `sqlite3` holds changes in a transaction until you call `conn.commit()`. When your code ends without it, the change is rolled back and the page says so under the output. Put `conn.commit()` before `conn.close()` at the end of your code. The SQL boxes do not have this problem: every Run saves the file.

You have broken your database

Reset database, at the bottom of the Database panel, puts it back the way it started: chapter 3 restores the original `employees.db`, and chapters 1 and 2 remove the file so you can build it again. Nothing is lost; the source data files are never modified.